

VirSME Application Performance Audit

Audit date: 17 July 2026 **Audit type:** Read-only source, configuration, and safe local build/startup audit **Repository:** `C:\Users\Nash Tech PC 03\Documents\Full stack` **Audience:** Product leadership, engineering, SRE, security, database, and cloud teams

This report distinguishes **measured facts**, **code-verified findings**, **strong indications**, **suspected risks**, and **measurement required**. No production system was accessed or load-tested. No secrets or personal records were read or reproduced. Builds were written only to isolated `C:\tmp\virsme-audit-*` directories.

1. Executive performance assessment

Overall performance maturity

VirSME's current performance maturity is **3/10 (early-stage, functionally broad but not performance-engineered for a GCC multi-tenant launch)**. The platform contains substantial operational capability, reasonable use of MongoDB projections/ `lean()` in parts of the backend, compound indexes in the communication models, static-upload cache headers, and some real-time room targeting. However, the delivered frontend payloads, synchronous in-process workload, unbounded tenant lists, per-request authentication queries, local document storage, single-process schedulers, lack of telemetry, and absence of deployment/load-test assets create material responsiveness, scalability, availability, and cost risks.

The highest-confidence user-facing limitation is frontend delivery. Every major React application eagerly imports its routes. Safe production builds emitted single entry chunks of **3.22–6.87 MB minified JavaScript** for People, CRM, Connect, Employee, and Admin; gzip sizes were **0.89–1.98 MB** before images. The Admin application additionally emitted **10.24 MB CSS (712.63 kB gzip)** because its Tailwind safelist generates a very large combinatorial set of utilities. These are verified build artifacts, not estimates.

Backend runtime and database latency could not be responsibly measured without a synthetic MongoDB dataset and isolated integration configuration. Code evidence nonetheless shows repeated DB work in authentication middleware, many unpaginated list paths, serial DB operations in loops, Puppeteer PDF generation inside HTTP requests, mailbox polling and cron jobs inside the web process, and Socket.IO handlers split across three `connection` registrations. These will prevent predictable horizontal scaling until separated or coordinated.

Strongest areas

- Communication schemas contain several query-aligned compound indexes, including inbox/sent/trash and time-ordered message indexes in `backend/src/models/AssignmentMessage.js`, `Chat.js`, and `WhatsAppMessage.js`.
- Many read queries already use projections and `.lean()`, reducing Mongoose hydration cost.
- Static uploads have seven-day immutable cache headers in `backend/src/index.js:190-223`.

- Communication lists use server-side pagination in several email paths, and frontend email list code uses a bounded page size of 50.
- Some independent dashboard and hierarchy queries use `Promise.all`, demonstrating an established pattern that can be extended.
- Authentication, encryption, permission checks, audit workflow history, and tenant scoping are present and must be preserved while optimising.

Most serious bottlenecks

1. **Oversized, eagerly loaded frontends:** no `React.lazy`/route splitting was found across seven frontends; entry JavaScript reaches 6.87 MB minified.
2. **Admin CSS explosion:** Tailwind safelist generates 10.24 MB minified CSS.
3. **Database work on every authenticated request:** employee authentication commonly performs employee plus session lookups; other middleware may query both user and employee/grant collections.
4. **Unbounded core lists and client-side pagination:** employees, CRM clients, and task paths fetch entire tenant datasets and populate related objects.
5. **Web process owns long-running and CPU-heavy work:** cron, IMAP, OpenAI calls, PDF/browser generation, email delivery, Socket.IO, and APIs share one Node process.
6. **No scaling-safe job ownership:** every web replica would run the same crons and potentially the same mailbox watcher.
7. **Local disk is application state:** 1,569 local uploaded/generated files consume 667.36 MB across the two upload roots, excluding document copies in other folders.
8. **No actionable observability:** 1,510 `console.*` call sites were counted, but no structured logger, tracing SDK, metrics dependency, or frontend RUM implementation was found.

Highest operational risks

- Duplicate cron execution and mailbox processing after horizontal scale-out.
- Single-instance dependency from local file storage and in-memory Socket.IO rooms/caches.
- Event-loop and memory pressure from Puppeteer, OCR/image libraries, spreadsheet parsing, and large response serialization.
- Tenant/noisy-neighbour exposure because resource usage, pagination, jobs, and monitoring are not bounded per tenant.
- Undetected regressions because the backend test script intentionally exits with “no test specified,” the two files named `test.js` are alternate server-sized files rather than a test suite, and no CI/infrastructure definitions were found.
- Socket room membership is accepted from client-provided identifiers in several handlers, and Socket.IO CORS is `origin: "*" ;` fixing this improves security and prevents abusive fan-out without weakening controls.

Highest-value optimisation opportunities

- Route-level code splitting plus dependency/chunk isolation: projected **60–85% reduction in initial JS transferred** on typical single-module entry routes.
- Replace the Admin safelist with an explicit bounded design-token map: projected **>90% reduction in Admin CSS raw size**.
- Paginate/project Employee, CRM, task, attendance, and reporting APIs; target **50–90% payload reduction** and bounded query/memory cost at large tenants.
- Introduce request-level auth context caching with short TTL and revocation-aware invalidation: remove **1–3 MongoDB reads from most authenticated requests**.
- Move scheduled, email, AI, document, payroll, and report work to an idempotent queue: protect API p95 latency and enable controlled worker scaling.

- Move files to Saudi/GCC-appropriate object storage behind a CDN and make the web tier stateless.
- Establish OpenTelemetry-compatible tracing, structured logs, RUM, Mongo slow-query capture, and per-tenant/AI cost metrics before major optimisation.

Areas that could not be measured

The following are **Measurement required**: authenticated page load/Core Web Vitals; API p50/p95/p99; MongoDB query plans, query counts, index sizes and cache ratios; production/development data volume; background-job duration/failure rate; AI latency/token/cost; concurrent-user capacity; Socket.IO connection capacity; error rate; availability; cache hit rates; regional latency from Saudi/GCC cities; production CPU/memory; deployment/rollback time; backup/restore performance; and cost per active tenant. No current hosting, database region, CDN, container, CI/CD, or multi-region definition exists in the repository.

2. Architecture and baseline

Concise architecture summary

VirSME is a multi-SPA system backed by a large Express/Mongoose monolith:

- **Frontends**: seven independent Vite/React 18/TypeScript SPAs: `admin_dashboard_hr`, `Employee_dashboard`, `People`, `complete-profile`, `Connect`, `CRM`, and `task_dashboard`. They use React Router, Tailwind/Radix UI, Axios, Socket.IO clients, and large document/chart/editor/export libraries. React Query providers exist, but no `useQuery` or `queryClient.*` call sites were found.
- **Backend**: Node.js CommonJS, Express 5, Mongoose 8, Socket.IO, node-cron, IMAP/SMTP, Puppeteer, Sharp/Jimp/Tesseract, PDFKit/pdf-lib, XLSX, and OpenAI. `backend/src/index.js` imports and mounts approximately 104 route files, 90 controllers, 96 models, schedulers, mailbox watcher, Socket.IO, TLS, and static storage.
- **Database**: shared MongoDB with tenant ownership mainly represented by `owner` ObjectIds. Tenant representation is inconsistent (`req.user._id`, `owner`, `createdBy`, and occasional arrays are normalised in middleware). Communication collections are relatively well indexed; Employee, SalarySlip, session, task, and several reporting paths are not fully aligned to observed filters/sorts.
- **State/integrations**: files are stored on local disk; Socket.IO uses the process-local adapter; cron and IMAP run in-process; no distributed cache or queue is declared. AI is used for CNIC vision extraction and general inbound-email classification.
- **Deployment**: application-level TLS and ports are embedded in `backend/src/index.js`; no Docker, Kubernetes, Terraform, serverless, reverse-proxy, CI/CD, or platform manifest was found. Actual hosting geography and topology are unknown.

Technology stack

Layer	Verified technology	Evidence
Web UI	React 18, Vite 5, TypeScript, React Router, Tailwind, Radix/shadcn	Seven <code>package.json</code> files and <code>src/App.tsx</code> entry points
Client data/network	Axios, Socket.IO client; React Query installed/providers only	App manifests; zero <code>useQuery</code> call sites
API	Node.js, Express 5.1	<code>backend/package.json</code> , <code>backend/src/index.js</code>

Layer	Verified technology	Evidence
Data	MongoDB via Mongoose 8.14	backend/package.json , 96 model files
Real time	Socket.IO 4.8	backend/src/index.js:519 , three connection blocks
Jobs	node-cron , timers, in-process loops	backend/src/index.js:2969-3268 , backend/src/schedulers
Documents/files	local disk, Multer, Puppeteer, PDFKit/pdf-lib, XLSX	routes/controllers and 667.36 MB local upload roots
Email	IMAP/imapflow/mailparser, Nodemailer/SMTP	backend/src/watcher.js , services
AI	OpenAI chat completions, image data URIs	backend/src/services/deepseekService.js
Observability	console logging and basic {ok:true} health only	1,510 console call sites; backend/src/index.js:316

Test environment and method

Item	Value
Host	Windows workspace; PowerShell; local date 17 July 2026
Node/npm	Node v25.2.1 , npm 10.2.4
Build data	Existing installed dependencies; production Vite builds; output isolated under C:\tmp
Runtime probe	Synthetic mongodb://127.0.0.1:1/virsme_audit , email receiver disabled, HTTP localhost port 49123; no real DB or external mailbox
Data	No application records loaded; source asset metadata only
Safety	No production access, external load, package installation, secret inspection, or application-code modification
Limitations	Node 25 may differ from the deployment runtime; Windows build times are not deployment times; bundle sizes are reliable for this source/dependency state

Existing measurements

Frontend production builds

Application	Wall time	Main JS minified	Main JS gzip	CSS minified	CSS gzip	Result
Admin HR	86.74 s	6,868.06 kB	1,979.61 kB	10,244.14 kB	712.63 kB	Success; 16,739 modules
Employee	32.07 s	5,129.72 kB	1,379.74 kB	287.55 kB	43.49 kB	Success; 4,918 modules
People	26.56 s	3,222.97 kB	891.56 kB	222.82 kB	34.43 kB	Success; 4,461 modules
Complete Profile	13.98 s	575.49 kB	172.94 kB	137.34 kB	39.65 kB	Success; 2,042 modules
Connect	31.99 s	4,903.63 kB	1,335.89 kB	260.65 kB	39.88 kB	Success; 4,308 modules

Application	Wall time	Main JS minified	Main JS gzip	CSS minified	CSS gzip	Result
CRM	22.60 s	3,542.93 kB	999.04 kB	234.49 kB	36.32 kB	Success; 3,841 modules
Task	10.12 s	508.39 kB	152.88 kB	97.39 kB	17.34 kB	Success; 2,062 modules

All seven builds warned about chunks over 500 kB where applicable. Admin also reported that `html2pdf.js` is both statically and dynamically imported, preventing chunk extraction. Large launcher PNGs are duplicated into multiple builds: CRM image 1,504.52 kB, People 670.07 kB, Connect/Task images about 607–623 kB.

Backend cold startup probe

The synthetic disconnected startup reached `/api/health` in **7,320 ms**, at **270.6 MB working set** and **5.047 CPU seconds**. This is a measured cold import/listen cost on Node 25 and Windows, with no working database. It does **not** measure readiness to serve authenticated traffic because the health route does not check MongoDB, queues, mailbox, storage, or AI dependencies.

Storage and tests

- `backend/src/uploads` : 324 files, 150.39 MB.
- `backend/uploads` : 1,245 files, 516.61 MB.
- Generated contract/NDA/certificate copies add 55 files (0.34 MB) in three additional source folders.
- `npm test` in `backend` exited 1 immediately with `Error: no test specified`. Frontend manifests have no test scripts.
- No CI or infrastructure definition files were found.

Missing measurements and assumptions

- **Measurement required:** all live latency, throughput, DB, job, AI, RUM, error, availability, deployment, regional, and cost baselines.
- Assumption: MongoDB is shared by tenants because models use an `owner` field; no database-per-tenant routing was found.
- Assumption: checked-in `dist` and `.env` changes predated the audit; they were not altered by audit builds.
- Assumption: the standalone Task app primarily uses `/api/chat/.../tasks` and `chatTask`; the older `/api/tasks` path appears schema-incompatible and requires ownership confirmation.

3. Performance scorecard

Area	Current rating	Main concern	Potential impact	Confidence
Frontend efficiency	2/10	0 lazy routes; 0.5–6.9 MB entry JS; Admin 10.2 MB CSS	Slow first load, parse/execute delay, high mobile data	Verified

Area	Current rating	Main concern	Potential impact	Confidence
User-perceived responsiveness	3/10	Large blocking payload plus dashboard request fan-out	Blank/loading time, interaction delay, unstable weak-network UX	Strongly indicated
Backend processing	4/10	7.32 s cold import; heavyweight synchronous/in-process concerns	Slow recovery/scaling and event-loop contention	Verified/strongly indicated
Database	4/10	Unbounded lists, query/index mismatch, per-request auth queries	Latency grows with tenant data; excess DB cost	Strongly indicated
API design	4/10	Inconsistent pagination/projection/timeouts and large body limits	High payload, memory pressure, unpredictable p95	Strongly indicated
Caching	3/10	Static upload caching only; no app/distributed auth/reference cache	Repeated DB/API work and higher latency/cost	Verified
Jobs/workflows	2/10	Cron/mail/PDF/AI share web process; no durable queue/DLQ	Duplicate/lost work and API degradation	Verified
AI performance/cost	3/10	Minimum 2,500 completion tokens for reasoning models; no usage metrics/cache	Avoidable latency/token cost	Verified
Infrastructure/deployment	2/10	No infra/CI definitions; local TLS and disk	Manual, slow, non-repeatable scaling/recovery	Verified
Scalability/multi-tenancy	2/10	Process-local state/jobs/files; inconsistent tenant context; no quotas	Noisy neighbours and unsafe horizontal scale	Verified
Reliability/resilience	3/10	Shallow health, no coordinated retries/circuit breakers/graceful shutdown	Cascading failures and risky deploys	Verified
Observability	1/10	1,510 console calls; no structured metrics/tracing/RUM	Long incident diagnosis and invisible regressions	Verified
Security/performance	3/10	No global rate limiter/Helmet; wildcard Socket.IO CORS/client room IDs	Abuse-driven resource exhaustion and data risk	Verified
Cost efficiency	3/10	Large transfer, repeated queries, local storage, untracked AI/log volume	Cost grows faster than tenants	Strongly indicated
Test/performance engineering	1/10	No functioning automated or load/performance suite	Regressions reach deployment	Verified

Ratings reflect present evidence, not missing production measurements. Missing evidence lowers confidence and operational maturity; it does not prove poor production latency by itself.

4. Detailed findings

ID	Area	Finding	Evidence	Root cause	User impact	Severity	Confidence
FE-01	Frontend	All seven SPAs eagerly load route/page modules; no route-level lazy loading exists.	<code>admin_dashboard_hr/src/App.tsx:9-77</code> ; analogous static imports in <code>Employee/People/Connect/CRM</code> ; repository search found zero <code>React.lazy / lazy()</code> sites.	Monolithic entry components and static page imports.	Every user downloads payroll, editors, PDF, charts, chat and other features before using one route.	Critical	Verified
FE-02	Frontend/CSS	Admin emits 10,244.14 kB CSS (712.63 kB gzip).	Measured build; broad regex safelist with <code>colors × shades × opacity × variants</code> at <code>admin_dashboard_hr/tailwind.config.ts:11-20</code> .	Combinatorial Tailwind safelist.	Added transfer, CSS parse/style cost, 86.74 s build.	High	Verified
FE-03	Frontend	Main JS is extremely large: Admin 6,868 kB, Employee 5,130 kB, Connect 4,904 kB, CRM 3,543 kB, People 3,223 kB.	Isolated production builds. Vite chunk warnings on all major apps.	Eager routes, broad dependency sets, static heavy exporters/editors, duplicated app shells.	Slow LCP/INP on GCC mobile/weak bandwidth; high memory.	Critical	Verified
FE-04	Frontend/data	React Query is shipped and providers wrap apps, but it performs no caching/deduplication.	<code>QueryClientProvider</code> in five major <code>App.tsx</code> files; zero <code>useQuery</code> and zero <code>queryClient.*</code> call sites.	Direct Axios/fetch remains the data-access convention.	Duplicate request fan-out, inconsistent stale/loading state, wasted bundle bytes.	High	Verified
FE-05	Dashboard	Admin/employee dashboards make many independent calls and duplicate roster retrieval.	Admin <code>Dashboard.tsx:474,490,507,552,615,641,1198-1201</code> ; <code>/employees</code> at 507 and 1201. Employee dashboard similarly fetches <code>/employees</code> , events, birthdays, balances, penalties, sessions and external weather.	Page-local effects with no shared query cache or composite bootstrap contract.	Longer workspace load and extra auth/DB work per request.	High	Strongly indicated

ID	Area	Finding	Evidence	Root cause	User impact	Severity	Confidence
FE-06	Lists	Employee pagination is client-side after the whole employee list is downloaded.	Backend unbounded <code>Employee.find(query)</code> at <code>routes/employees.js:124-128</code> ; frontend <code>filtered.slice(...)</code> at <code>admin_dashboard_hr/src/pages/Employees.tsx:719</code> .	API lacks cursor/page contract.	Payload, sorting, filtering and memory grow linearly per tenant.	High	Verified
FE-07	Rendering	Several UI/controller files are exceptionally large and render/map complex lists without virtualisation.	<code>Employee_dashboard/src/pages/Whatsapp.tsx</code> 321.6 kB; CRM <code>EmailDetail.tsx</code> 315.7 kB; Admin <code>EmployeeEdit.tsx</code> 234.3 kB; attendance maps records at <code>Attendance.tsx:1765</code> ; no react-window/virtual list dependency found.	Feature accumulation in monolithic components.	Long main-thread tasks, rerender risk, difficult profiling/maintenance.	High	Strongly indicated
FE-08	Assets	Launcher images are large and repeated across builds.	Build artifacts: CRM PNG 1,504.52 kB; People 670.07 kB; task/connect images 607–623 kB.	Unoptimised PNGs bundled per SPA.	Avoidable initial/network transfer and cache fragmentation across subdomains.	Medium	Verified
FE-09	Build quality	Builds report ineffective dynamic import and duplicate JSX/case warnings.	Admin/Employee/People: <code>html2pdf.js</code> static import prevents dynamic splitting; Connect duplicate <code>onPaste</code> ; CRM/People duplicate switch cases.	Mixed import conventions and no warning-as-error quality gate.	Lost code-splitting benefit and latent behaviour defects.	Medium	Verified
BE-01	Startup/resource	Backend cold import/listen is heavy: 7.32 s, 270.6 MB working set, 5.047 CPU s without a DB.	Synthetic local probe; <code>index.js</code> imports 104 routes plus Puppeteer, schedulers, watcher, models and Socket.IO.	Single eager composition root loads every capability.	Slow deploy/rollback/autoscale recovery; high idle memory.	High	Verified
BE-02	Authentication/DB	Authenticated requests usually perform DB reads before controller work.	<code>middleware/auth.js:30-71</code> ; <code>empAuth.js:47-89</code> performs employee and session queries; <code>unifiedAuth.js:34-79</code> ; write paths in <code>anyPayrollAuth.js:39-132</code> .	DB-backed identity/session/permission reconstruction on every call.	Adds DB round trips to every dashboard request and amplifies fan-out.	High	Verified

ID	Area	Finding	Evidence	Root cause	User impact	Severity	Confidence
BE-03	API/network	No global response compression middleware is configured.	backend/package.json has no compression dependency; index.js body/static setup has none.	Compression delegated nowhere visible in repository.	Large JSON responses consume bandwidth unless an unknown proxy compresses them.	High	Verified
BE-04	External dependencies	Consistent outbound timeouts, aborts, bounded retries and circuit breakers are absent.	Only isolated timeouts in geo/email/watcher; no shared HTTP client policy.	Direct Axios/OpenAI/SMTP/IMAP calls from features.	Hung requests/workers and retry storms during dependency failure.	High	Strongly indicated
DB-01	Employee queries	Employee tenant/list/search filters lack query-aligned compound indexes.	Employee indexes are email, isTrashed, trashedAt, blockedUsers, supervisionMode; list filters owner, status, department, role, name sort/search at routes/employees.js:77-128.	Indexes added for isolated fields rather than observed tenant query shapes.	Tenant roster/dashboard queries can scan and sort in memory.	High	Strongly indicated
DB-02	Payroll	SalarySlip defines no indexes despite owner/employee/month/year access patterns.	models/SalarySlip.js:5-80 contains no .index; queries use employee/month/year in salary routes.	Missing query-to-index review.	Payroll lookup and report generation degrade as history grows.	High	Verified
DB-03	CRM/tasks	CRM/client and legacy task lists are unbounded and hydrated with populates.	clientInfoController.js:107-110,175-178,265-268; tasksController.js:207-234.	No default pagination/field projection and Mongoose document hydration.	Large customers cause growing DB, memory and response cost.	High	Verified
DB-04	Task correctness/performance	/api/tasks controller expects client, assignedTo, and attachments, but models/Task.js requires workspace, space, and uses assignees.	tasksController.js:152-234; models/Task.js:14-65. Standalone Task uses /api/chat/.../tasks instead.	Two task implementations evolved against one model name.	Legacy journey may fail or issue ineffective queries; optimisation is unsafe until ownership is resolved.	High	Verified
DB-05	N+1/serial work	DB and crypto operations occur serially inside loops.	assignmentMessageController.js:5187-5402, crmAccess.js:61, leaveSyncCron.js:36-49, salary decrypt loop salarySlips.js:268, PDF page loops.	Row-at-a-time workflow implementation.	Payroll, inbox counts, access grants, cron and documents slow linearly.	High	Verified

ID	Area	Finding	Evidence	Root cause	User impact	Severity	Confidence
DB-06	Search	Employee search uses case-insensitive unanchored regex across four fields.	<code>routes/employees.js:106-120</code> .	No normalised search field/text/search service.	Collection scans under broad search terms; regex abuse risk.	Medium	Verified
API-01	API consistency	Pagination, response envelopes and auth middleware differ across modules.	Employee/CRM/tasks are unbounded; email uses varying pagination, total, pages shapes handled defensively in <code>EmailList.tsx:1641-2053</code> .	Module-local API evolution.	Client complexity, duplicate calls, difficult SLO instrumentation.	Medium	Verified
CACHE-01	Caching	Only static-upload caching is explicit; no distributed/app/reference/auth cache exists.	<code>STATIC_CACHE_OPTS</code> in <code>index.js:190</code> ; no Redis/cache dependency; React Query unused.	Caching not designed as a cross-cutting capability.	Repeated stable queries and permission/session lookups increase latency/cost.	High	Verified
JOB-01	Workflows/scaling	Cron schedules execute inside every web process.	Four schedules in <code>index.js:2969, 3006, 3152, 3241</code> ; additional scheduler modules imported at <code>index.js:186-188</code> .	No durable queue or singleton scheduler/lease.	Horizontal replicas duplicate payroll/leave/message work.	Critical	Verified
JOB-02	Mailbox	Importing <code>watcher.js</code> opens another Mongoose connection path, then <code>startWatcher()</code> is called after main DB connection; each reconnect arms a new polling interval without retaining/clearing its handle.	<code>watcher.js:34-40</code> , <code>index.js:383-390</code> , <code>watcher.js:793</code> .	Integration owns process-global connection/timer lifecycle.	Extra connections, duplicate polling after reconnects, memory/work duplication.	High	Verified
JOB-03	Documents	Puppeteer browsers/pages and PDF merging run inside HTTP handlers and page loops.	<code>routes/docs.js:1258-1267, 1589-1628, 2327-2352</code> .	Document generation is synchronous request work.	Request timeouts, high memory/CPU, noisy-neighbour spikes.	Critical	Verified
AI-01	AI cost/latency	Reasoning-model calls force at least 2,500 completion tokens even when callers request 10 or 300.	<code>deepseekService.js:17-23</code> ; email classifier calls with 10 at 136-144.	Shared helper conflates reasoning and short classification budgets.	Excess latency and potential token spend for deterministic short labels.	High	Verified
AI-02	AI governance	No request hash cache, usage/token/cost metrics, latency metrics, explicit retry/backoff/timeout, or audit fields were found.	<code>deepseekService.js</code> ; only process-lifetime model fallback flag.	Thin direct SDK wrapper.	Repeated cost, unbounded waits, no tenant/workflow cost attribution.	High	Verified

ID	Area	Finding	Evidence	Root cause	User impact	Severity	Confidence
AI-03	Deterministic logic	Every non-empty inbound email is sent to an LLM for one of five labels, although high-confidence acceptance/rejection/leave patterns can be deterministic before fallback.	<code>watcher.js</code> classification path and <code>deepseekService.js:121-151</code> .	Regex classifier was fully replaced rather than retained as a confidence gate.	Avoidable AI latency/cost on common messages.	Medium	Verified
INF-01	Storage/scaling	Files are stored on local web-server disk and served by Express from two roots.	<code>index.js:190-223</code> ; 667.36 MB/1,569 files measured; Multer disk storage in tasks.	No object-storage abstraction/CDN manifest.	Prevents stateless replicas; backup/restore and cross-region delivery risk.	Critical	Verified
INF-02	Deployment	No CI/CD, container, infrastructure-as-code, reverse-proxy, or cloud manifest exists; TLS is handled in application code.	Repository inventory; <code>index.js:456-516</code> .	Host-specific/manual deployment architecture.	Slow, inconsistent deploys and rollback; unclear Saudi/GCC topology.	High	Verified
SCALE-01	Real time	Socket.IO uses process-local rooms and three <code>io.on("connection")</code> blocks; no shared adapter is declared.	<code>index.js:519,581,2389,2761</code> ; no Redis adapter dependency.	Real-time features accumulated in one file without scale topology.	Duplicate listeners per socket and missed messages across replicas.	Critical	Verified
REL-01	Readiness/resilience	<code>/api/health</code> always returns <code>{ok:true}</code> and startup begins listening before DB readiness; graceful shutdown is incomplete.	<code>index.js:316,383-516</code> ; only watcher SIGINT handling found.	Liveness used as readiness; lifecycle not orchestrated.	Traffic can hit an unready instance; unsafe deploy termination.	High	Verified
OBS-01	Observability	1,510 console call sites, no structured logger, tracing SDK, metrics client, or RUM.	Static count; manifests.	Debug logging substituted for telemetry.	Cannot locate p95 bottlenecks, tenant hot spots, or regressions.	Critical	Verified
SEC-01	Security/performance	No global rate limiter/Helmet; Socket.IO allows wildcard origin and several joins trust client-provided IDs.	<code>index.js:519-530,581+</code> ; no Helmet/rate-limit dependencies.	Perimeter controls absent from app/repository.	Abuse can create DB/CPU/socket fan-out; security incident also becomes performance outage.	Critical	Verified
SEC-02	Resource limits	JSON/form bodies allow 10 MB globally; task files allow 20 MB each; expensive search/document/AI routes lack visible route-specific quotas.	<code>index.js:225-226</code> ; <code>tasksController.js:54-57</code> .	Broad global defaults, no tenant/user budget.	Memory/CPU exhaustion and storage growth.	High	Verified

ID	Area	Finding	Evidence	Root cause	User impact	Severity	Confidence
TEN-01	Multi-tenancy	Tenant identity/scoping is inconsistently normalised and there are no per-tenant quotas or telemetry.	Auth middleware alternates <code>_id</code> , <code>owner</code> , <code>createdBy</code> , arrays; models/query paths vary.	Incremental shared-database design without central tenant context.	Isolation defects, noisy neighbours, hard capacity/cost attribution.	Critical	Strongly indicated
TEST-01	Quality/performance	No functioning automated, benchmark, or load-test suite and no CI gate.	Backend <code>npm test</code> exits 1; frontend scripts omit tests; no infra/CI files.	Delivery focused on feature implementation.	Performance and correctness regressions cannot be prevented.	Critical	Verified
REG-01	GCC readiness	Hosting region, DB region, CDN, residency, localisation load and cross-region DR cannot be verified.	No deployment definitions or runbooks in repository.	Architecture is external/manual or undocumented.	Unknown Saudi/GCC latency, residency compliance and RTO/RPO.	High	Measurement required

5. Optimisation register

Expected improvements below are **engineering projections/targets**, not measured current gains. Validate every item before rollout.

ID	Recommendation	Implementation steps	Expected improvement	Effort	Risk	Dependencies	Priority	Validation method
OPT-01	Split every SPA by route and heavy feature	Convert route imports in each <code>src/App.tsx</code> to <code>lazy(() => import(...))</code> ; add Suspense route skeletons; isolate payroll/PDF/XLSX/editor s/charts/chat; ensure <code>html2pdf.js</code> is never statically imported on initial paths.	60–85% lower initial JS on common entry routes; lower parse/heap.	M	Medium	Bundle budget tooling	P0	Compare Vite manifest/gzip; Lighthouse/RUM LCP and INP on login/workspace routes.
OPT-02	Replace Admin Tailwind regex safelist	Map supported semantic colors to a finite literal class list or CSS variables; remove shade/opacity/variant Cartesian product; add CSS budget.	>90% raw Admin CSS reduction; build under 45 s target.	S	Low	Visual regression screenshots	P0	Build CSS <250 kB gzip <50 kB; screenshot all status/theme variants.

ID	Recommendation	Implementation steps	Expected improvement	Effort	Risk	Dependencies	Priority	Validation method
OPT-03	Establish bundle budgets and asset optimisation	CI fail thresholds: initial JS <=250 kB gzip per route, CSS <=75 kB gzip, initial images <=300 kB; convert launcher PNGs to responsive WebP/AVIF/SVG; share via CDN origin.	1–4 MB less first-visit transfer depending app.	S-M	Low	CI and CDN	P0	Vite stats plus cache-disabled network trace at 4G/150 ms RTT.
OPT-04	Adopt React Query for shared server state	Central query keys include tenant/user; migrate /auth/me , employee names, company profile, shifts, payroll period, notifications and dashboard calls; set deliberate staleTime ; invalidate on mutation/socket events.	Deduplicate 20–50% of repeated workspace requests; consistent loading/retry.	M	Medium	Tenant context contract	P0	Browser request count, cache hit/dedup counters, mutation consistency tests.
OPT-05	Add cursor pagination/projection to core lists	Add limit (default 50, max 200), stable cursor (createdAt, _id or normalised name, _id), explicit fields and totals only when requested to employees, CRM clients, tasks, attendance, requests and reports; update UIs to server pagination/virtual lists.	Bounded DB/heap/payload; 50–90% lower large-list payloads.	L	Medium	API version/compatibility	P0	Synthetic 1k/10k/100k rows; explain plans; payload and p95 budgets.

ID	Recommendation	Implementation steps	Expected improvement	Effort	Risk	Dependencies	Priority	Validation method
OPT-06	Create query-index review and initial index set	After capturing real query shapes, test: Employee {owner:1,isTrashed:1,status:1,name:1} for roster; SalarySlip {owner:1,employee:1,year:1,month:1} unique if business rule permits; Session {employeeId:1,active:1,loginTime:-1}; Task/ChatTask indexes matching actual active path. Remove redundant indexes only after usage evidence.	Target p95 <50 ms for indexed point/list queries at representative scale.	M	Medium	Staging dataset/Profiler	P0	explain('executionStats'); examined/returned ratio <10; measure index bytes/write latency.
OPT-07	Cache auth/session/permission context safely	Create cache key authctx:{tenant}:{subject}:{tokenVersion}; value minimal identity/permission/session-active metadata; TTL 30–60 s; invalidate on logout, role/permission/session/device changes; never cache bearer tokens; tenant prefix mandatory.	Remove 1–3 DB reads from most authenticated requests; lower DB p95/cost.	M	Medium	Redis/managed cache	P0	Middleware DB query count and auth p95; revocation tests must pass within defined bound.
OPT-08	Move cron/workflows to durable queue	Introduce queue with separate scheduler and workers; job keys/idempotency; tenant and priority; retries with jitter; DLQ; leases; move scheduled messages, leave sync/reset, absence backfill, notifications, reports, payroll batches, document generation, AI/email tasks.	No duplicate replica cron; API p95 isolation; recoverable workflows.	L	Medium	Redis/managed queue or cloud queue	P0	Kill/retry/duplicate tests; exactly-once business effect; queue lag SLO.

ID	Recommendation	Implementation steps	Expected improvement	Effort	Risk	Dependencies	Priority	Validation method
OPT-09	Move files to object storage/CDN	Define FileStore interface; direct presigned multipart upload; MIME/magic-byte/size scan; immutable versioned keys tenant/{tenantId}/{type}/{uuid}; private signed downloads; lifecycle tiers; migrate/verify checksums.	Stateless web tier, lower egress/CPU, horizontal scale; faster GCC delivery.	L	Medium	Saudi-residency decision, KMS, CDN	P0	Parallel checksum migration, signed access tests, regional download TTFB, backup restore drill.
OPT-10	Separate web, real-time and worker concerns	Keep API stateless; run Socket.IO with authenticated handshake and shared adapter; run workers separately; stop importing Puppeteer/watcher in API boot path; lazy-load rare processors.	Cold start <2 s target; lower API idle memory and noisy-neighbour risk.	L	Medium	Queue/cache/container platform	P1	Startup/RSS benchmarks; cross-replica socket tests; worker saturation test.
OPT-11	Consolidate Socket.IO connection/auth flow	One connection handler; verify JWT during handshake; server-derived tenant/user rooms only; remove client-chosen privileged room joins; bounded event payload/rate; adapter and backpressure metrics.	Removes duplicate listener work; secure predictable fan-out.	M	Medium	Shared adapter	P0	Listener count per socket, unauthorised join tests, 10k-connection soak in staging.
OPT-12	Add compression and HTTP cache policy at edge	Prefer managed reverse proxy/CDN Brotli; fallback Express compression for JSON/text; hash static assets with one-year immutable; HTML no-cache; ETag /conditional GET for reference data.	60–85% lower JSON/text bytes where currently uncompressed.	S	Low	Edge/proxy ownership	P0	Content-Encoding, transferred bytes, CPU comparison; exclude already-compressed files.

ID	Recommendation	Implementation steps	Expected improvement	Effort	Risk	Dependencies	Priority	Validation method
OPT-13	Build shared outbound dependency client policies	Per dependency connect/read/overall timeouts; AbortController; bounded exponential backoff with jitter only for idempotent operations; circuit breaker; bulkheads; retry budgets; webhook idempotency keys.	Prevent hung requests and cascading failure; stable p99.	M	Medium	Observability	P1	Fault injection for OpenAI/SMTP/IMAP/weather; verify bounded latency and fallback.
OPT-14	Make PDF/report/payroll generation asynchronous	API validates/enqueues and returns 202 + jobId ; worker pools reuse a bounded browser where safe, cap pages/concurrency/memory, store result in object storage, notify completion.	API response <300 ms; prevents multi-second browser work blocking API.	M-L	Medium	Queue/object storage	P0	20 concurrent report jobs; API p95 remains within SLO; memory ceiling and retry tests.
OPT-15	Fix AI routing, budgets and caching	Use deterministic rules for obvious intents; use a low-latency classifier for ambiguous email; remove forced 2,500 completion minimum; structured JSON output; downscale/compress CNIC images; request timeout; usage logging; cache by tenant + workflow + model + promptVersion + content SHA-256.	50–90% lower classifier token cost; 20–60% lower common-case latency.	M	Medium	AI evaluation set/security review	P0	Labeled synthetic email/CNIC evaluation; p50/p95 tokens, latency, accuracy, cost per workflow.
OPT-16	Instrument end to end	Structured JSON logger with request/trace/tenant IDs (no PII); OpenTelemetry HTTP/Mongoose/queue/external spans; RED/USE metrics; Mongo slow query; RUM Web Vitals; AI tokens/cost; dashboards/alerts.	Diagnosis from hours to minutes; evidence for next optimisations.	M	Low	Telemetry backend/data policy	P0	Trace one required journey UI→API→DB/job; alert and sampling tests; PII review.

ID	Recommendation	Implementation steps	Expected improvement	Effort	Risk	Dependencies	Priority	Validation method
OPT-17	Introduce readiness, shutdown and deployment safety	/live process-only; /ready checks Mongo/queue/cache/storage with short timeouts; stop accepting traffic, drain HTTP/socket/jobs, close connections; rolling/canary deploy; automated rollback.	Avoid unready traffic and lost work; zero-downtime target.	M	Low	Container/orchestrator	P0	Termination/drain test under traffic; readiness dependency failure test.
OPT-18	Add abuse/resource controls	Helmet/CSP tuned per SPA; per-IP + per-user + per-tenant rate limits; route-specific body/upload limits; file magic/type scanning; query filter allowlists and max date ranges; socket event budgets.	Prevent abuse-driven outages without weakening auth/encryption.	M	Low-Medium	Shared rate-limit store	P0	Security performance tests: 429 behaviour, bounded memory, legitimate peak not throttled.
OPT-19	Centralise tenant context and quotas	Middleware derives immutable tenantId server-side; repositories require it; compound indexes start with tenant; per-tenant API/job/socket/storage/AI budgets; tiered limits; tenant-aware metrics.	Stronger isolation and predictable noisy-neighbour control.	L	Medium	Data model migration	P0	Cross-tenant negative tests; query lint/review; quota fairness load tests.
OPT-20	Resolve duplicate task domains	Determine canonical Task journey (ChatTask vs Task); migrate or retire /api/tasks ; align schema/controller/indexes; add compatibility tests.	Removes failing/dead query path and duplicated maintenance.	M	Medium	Product/data migration decision	P0	Contract tests for create/assign/update/list across roles and migrated records.
OPT-21	Optimise dashboard bootstrap	Create versioned /api/workspace-bootstrap? modules=... or use HTTP/2 parallel calls with React Query; return minimal profile, permissions, tenant branding, counts and references; do not embed large lists.	Reduce startup requests/auth queries by 30–60%; faster first useful content.	M	Medium	Query cache/auth context	P1	Request waterfall and server span comparison; payload <100 kB gzip.

ID	Recommendation	Implementation steps	Expected improvement	Effort	Risk	Dependencies	Priority	Validation method
OPT-22	Normalise searchable fields/search service	Store lowercased email/phone/name prefixes; allow exact/prefix filters; minimum query length; escape regex; optionally use managed Mongo search for ranked cross-module search.	Avoid collection scans; search p95 <300 ms target.	M	Medium	Index/storage cost	P1	100k synthetic records; explain plans; relevance/security tests.
OPT-23	Reduce log cost safely	Replace console calls with levels/sampling; redact tokens, auth headers, emails and document fields; avoid request body/header dumps; retention by severity; audit log remains durable/separate.	Lower I/O/storage and leak risk; clearer incidents.	M	Low	Structured logger	P1	Log bytes/request, redaction unit tests, audit completeness.
OPT-24	Establish Saudi/GCC deployment decision	Measure Riyadh/Jeddah/Dammam/Dubai/Doha/Kuwait/Manama/Oman synthetic latency; select Saudi-resident primary app/DB/storage if required; CDN static assets; same-region app/cache/DB; encrypted cross-region DR only after residency/legal approval.	Target GCC API network RTT <80 ms for primary markets; lower asset latency.	L	Medium	Legal/data-residency review/provider selection	P1	Regional probes, residency evidence, failover/restore exercise.
OPT-25	Create automated quality/performance gates	Unit/contract/integration tests; seeded synthetic Mongo; bundle budgets; API benchmark smoke; migration/index tests; CI parallel build/cache; deploy provenance.	Prevent recurrence; faster safe releases.	M	Low	CI platform	P0	Green pipeline required; reproducible clean build; regression intentionally caught.

ID	Recommendation	Implementation steps	Expected improvement	Effort	Risk	Dependencies	Priority	Validation method
OPT-26	Data lifecycle and maintenance	Per-collection retention; archive old chats/audit/report artifacts appropriately; TTL only for truly disposable sessions/tokens; online index builds; backup/PITR/restore tests; object lifecycle tiers.	Bounded storage/index working set and backup duration.	M	Medium	Legal retention policy	P2	Growth forecast, restore timing, archived-query acceptance tests.

Cache designs

Caching must never bypass authorisation. Every shared key starts with tenant ID; sensitive values are encrypted in transit/at rest; invalidation events are audited where access decisions change.

Cache	Key and value	Expiry	Invalidation	Tenant isolation / stale risk	Expected benefit
Auth context	<code>authctx:{tenant}:{subject}:{tokenVersion}</code> → minimal role, permissions, active-session marker	30–60 s	Logout, password/token version, role/permission, session/device change	Mandatory tenant prefix; stale permission window must be ≤TTL or push invalidation	Remove 1–3 DB reads/request
Reference data	<code>ref:{tenant}:departments:v{version}</code> , similarly shifts/designations/settings	5–30 min	Transaction increments tenant reference version; pub/sub invalidation	Low stale risk except payroll/attendance settings—use 1–5 min there	Faster forms/dashboards; fewer repeated reads
Company profile/branding	<code>company:{tenant}:profile:v{updatedAt}</code> → public/minimal branding fields	30 min	Profile update	Do not cache private bank/tax fields in public variant	Removes repeated header/bootstrap queries
Employee name picker	<code>employees:{tenant}:active-names:v{rosterVersion}</code> → <code>_id,name,avatar,role</code> only	5 min	Join/offboard/name/role update	Never include payroll/PII	Smaller, reusable picker payload
Permission decision	<code>perm:{tenant}:{subject}:{resource}:{action}:{permVersion}</code> → boolean/reason code	30 s	Permission/access grant mutation	Deny by default on cache failure; no cross-tenant keys	Reduces repeated grant lookups
Dashboard aggregates	<code>dash:{tenant}:{role}:{period}:{aggVersion}</code> → counts only	30–120 s	Relevant mutation event or short TTL	Show “updated at”; never cache user-specific data under role-only key	Faster dashboards and bounded report queries

Cache	Key and value	Expiry	Invalidation	Tenant isolation / stale risk	Expected benefit
Email/chat first page	Client React Query key [tenant,user,'threads',filter,page,limit]	stale 15–30 s	Socket event/mutation invalidates affected thread/filter	User ID mandatory; brief stale badge acceptable	Avoid duplicate UI calls, immediate cached navigation
Search suggestions	search:{tenant}: {subjectScopeHash}:{type}: {normalisedQuery}	30–60 s	TTL; roster/client version	Scope hash prevents privilege leakage	Faster typeahead, DB protection
Generated document	document:{tenant}: {templateVersion}: {dataVersion}:{locale}: {hash} → object key	Retain per legal policy	Employee/payroll/template version changes produce new key; immutable	Private signed URL; do not reuse across tenant/user scope	Avoid repeat Puppeteer generation
AI classification	ai:{tenant}:email-intent: {model}:{promptVersion}: {contentSha256} → label/confidence/usage	7–30 d	New prompt/model creates new namespace; deletion request purges	Hash input; restrict cached value, encrypt; stale risk low for immutable email	Eliminate repeated classification
AI CNIC extraction	ai:{tenant}:cnic:{model}: {promptVersion}: {imageSha256} → validated fields	Short, policy-defined (0–24 h preferred)	Purge immediately after onboarding completion/retention event	Highly sensitive; consider no distributed cache—idempotency record instead	Prevent retry duplication while minimising PII retention
CDN assets	Content-hashed JS/CSS/image URL → immutable bytes	1 year	New hash on change	Public assets only	High hit rate, low regional latency

6. Quick wins

1. Replace the Admin Tailwind safelist with finite semantic classes and add a CSS budget.
2. Lazy-load top-level routes and the payroll/PDF/XLSX/editor/chart modules first.
3. Optimise the 1.5 MB CRM launcher PNG and other launcher images; serve hashed assets with long cache headers.
4. Add edge Brotli/gzip for JSON/JS/CSS and verify it rather than assuming it.
5. Add default/max pagination and projections to `/api/employees`, CRM clients, tasks, requests and attendance.
6. Remove unused React Query providers/dependency immediately if migration is deferred; preferably migrate the duplicated `/auth/me`, company profile and employee-name calls.
7. Add database indexes only after staging `explain` confirms the observed query shapes; start with SalarySlip and Employee roster candidates.
8. Consolidate the three Socket.IO connection blocks and authenticate the handshake.
9. Add route-specific rate limits and lower default body limits; preserve strong auth/encryption/audit logging.
10. Add structured request timing, request/tenant correlation, DB query timing and AI usage metrics before deeper tuning.

7. Prioritised roadmap

Immediate critical actions (week 0–2)

- Implement OPT-16 instrumentation skeleton and capture a synthetic baseline.
- Apply OPT-02 Admin CSS fix, OPT-01 first-route splits, and OPT-03 budgets.
- Add bounded pagination/projection to the largest unbounded lists (OPT-05).
- Lock down/rationalise Socket.IO and resource limits (OPT-11/18).
- Stop any plan to horizontally scale the current process until jobs/mailbox have singleton leases or are disabled per replica (OPT-08).
- Decide the canonical Task model/path (OPT-20).

First 30 days

- Deploy React Query migration for shared bootstrap/reference calls (OPT-04/21).
- Add auth-context cache with revocation tests (OPT-07).
- Capture query profiler/explain evidence and deploy only proven indexes (OPT-06).
- Establish CI tests, clean builds, bundle budgets and API smoke benchmarks (OPT-25).
- Add readiness/graceful shutdown and compressed edge delivery (OPT-12/17).
- Create provider-neutral queue, object-storage and regional decision records.

Days 31–60

- Move document/report/notification/scheduled-message work to workers (OPT-08/14).
- Migrate new uploads to object storage and proxy old files through the abstraction (OPT-09).
- Separate API composition from rare heavy imports; reduce startup/RSS (OPT-10).
- Implement outbound timeout/retry/circuit policies (OPT-13).
- Implement AI deterministic gate, correct token budgets and cost/latency telemetry (OPT-15).

Days 61–90

- Complete server-side pagination/virtualisation across attendance, CRM, tasks, approvals, payroll and reports.
- Migrate existing files with checksums and enable CDN/signed delivery.
- Deploy shared Socket.IO adapter and cross-replica tests.
- Add per-tenant quotas/cost dashboards and central tenant context (OPT-19).

- Run normal, peak and stress benchmarks in the isolated staging environment.

Three to six months

- Deploy Saudi/GCC primary architecture based on measured latency and residency review.
- Add autoscaling by API saturation, queue lag and socket connections—not CPU alone.
- Implement retention/archive/PITR/restore drills and cost-per-tenant reporting.
- Modularise the largest frontend components and duplicated app shells into versioned shared packages/micro-frontends only where release independence justifies it.

Longer-term architectural improvements

- Keep a modular monolith for transactional business logic initially; split **workers**, **real-time gateway**, **document service** and **search** where scaling characteristics materially differ. Avoid service proliferation before telemetry proves need.
- Consider read models/materialised aggregates for dashboards and reporting after query evidence, not blanket denormalisation.
- For large tenants, evaluate MongoDB sharding/tenant placement only after indexed single-cluster limits are measured. A tenant-first shard key has isolation benefits but can create hot tenants; hashed tenant distribution complicates tenant moves and range reports.
- Add secondary region DR only after Saudi residency, encryption, RPO/RTO and cost decisions. Active-active business writes are not recommended until conflict semantics exist.

8. Performance targets

These are **recommended targets, not current measurements**. Segment by app, route, device, tenant size and region.

Metric	Recommended target
Public/login LCP	p75 <=2.0 s on mid-tier mobile, 4G, 150 ms RTT
Authenticated workspace LCP	p75 <=2.5 s; cached navigation <=1.0 s
INP / CLS	INP p75 <=200 ms; CLS p75 <=0.1
Initial route transfer	JS <=250 kB gzip; CSS <=75 kB gzip; total critical transfer <=500 kB excluding user content
API latency	reads p50 <=100 ms, p95 <=300 ms, p99 <=800 ms; writes p95 <=500 ms excluding async work
Database	point/indexed p95 <=20 ms; list/aggregate p95 <=100 ms; examined/returned <=10 for key queries
Payload	normal list <=250 kB gzip; page default 50/max 200; document data via object storage
Error rate	HTTP 5xx <0.2% over 5 min; critical workflows <0.1% final failure
Availability	99.9% initial launch; 99.95% after redundancy/operational maturity

Metric	Recommended target
Jobs	notifications p95 <=30 s; scheduled messages <=60 s late; standard reports p95 <=2 min; payroll batch target set by employee bands
AI	classifier p95 <=2 s, vision p95 <=10 s; explicit timeout <=15/30 s; fallback success >=99%; tokens/cost recorded 100%
Concurrent users	prove 2x forecast peak with >=30% headroom; initial test bands 100/500/1,000 and socket bands 1k/5k/10k
Startup/deploy	API ready <=2 s warm image / <=5 s cold; zero-downtime; rollback <=10 min
Build	each SPA <=45 s on CI reference runner; full parallel pipeline <=10 min
Resources	API steady-state RSS <=200 MB target after separation; CPU <60% at expected peak; no sustained event-loop lag >50 ms
Cache	reference/auth cache hit >=80% after warm-up; invalidation correctness 100% in tests
Cost	track monthly compute, DB, storage, egress, email and AI per active tenant/user; set tier margin target after 30-day baseline

9. Benchmark and load-testing plan

Safe environment and tools

Create a production-shaped, isolated staging account/region with synthetic tenants only, separate Mongo/queue/cache/object-store/email sandbox/AI test budget, and no production network route. Use existing tooling where possible; package/tool additions require approval. Recommended candidates: k6 or Artillery for HTTP/WebSocket, Playwright/Lighthouse for journeys/RUM lab, Mongo profiler/ `explain`, OpenTelemetry collector, and provider-native resource metrics.

Synthetic data bands

Band	Tenants	Employees/tenant	Clients/tenant	Tasks	Messages	Attendance years	Files
Small	20	25	100	5k total	100k	1	5k
Normal	100	100	500	100k	2m	3	100k
Large tenant	1 + normal mix	5,000	50,000	1m	10m	5	1m metadata objects

All identities, documents, emails, CNIC-like images and payroll values must be generated and non-real. Preserve realistic distributions, tenant skew, attachments, date ranges and approval depth.

Journeys to test

Login/bootstrap; employee profile/list/search; onboarding; attendance/leave; task create/assign/update; CRM list/search/update; email/chat/WhatsApp send/read; payroll calculation/slips; expense submit/approve; performance review when implementation is identified; dashboards/reports; upload/download; cross-module approval; AI classify/CNIC; reconnect and socket fan-out.

Load profiles

Scenario	Ramp and duration	Concurrency/load	Success criteria	Failure criteria
Smoke	1→10 users in 2 min, hold 5 min	10	0 functional errors; traces complete	Any data-isolation/correctness error
Normal	0→100 in 10 min, hold 30 min	70% reads, 20% writes, 10% realtime	API/DB targets; <0.2% 5xx	SLO breached 5 min or queue lag grows continuously
Expected peak	0→500 in 20 min, hold 60 min	Payroll/report spike included	p95 within targets; CPU <70%; DB headroom >=30%	p99 >2× target, errors >1%, resource saturation
Large tenant/noisy neighbour	Normal mix plus one 5k-employee tenant	Tenant generates 40% load	Other tenants p95 degrades <20%; quotas work	Cross-tenant data or >50% latency impact
Socket soak	1k→5k→10k connections, 2 h	Realistic rooms/events/reconnect	No listener/memory leak; delivery >=99.9%	RSS grows unbounded, missed/duplicate event >0.1%
Job peak	Payroll month end + reports + messages	Worker concurrency stepped	API SLO maintained; queue drains in target	Duplicate business effect, DLQ unexplained, API saturation
Stress	Increase users 20% every 10 min	Until SLO break	Graceful 429/backpressure; recovery in 10 min	Crash, data corruption, retry storm
Endurance	50–70% peak for 8–24 h	HTTP + sockets + jobs	Stable RSS/connections/lag	>10% unexplained resource drift or timer/listener growth
Dependency fault	Inject DB/cache/queue/AI/email latency/errors	During normal load	Bounded timeout, breaker, graceful degradation	Hung requests, duplicate writes, cascading retries

Metrics to capture

- Browser: DNS/connect/TLS/TTFB, request count/bytes, LCP/INP/CLS, long tasks, JS heap, errors.
- API: rate/errors/duration by route/status/tenant tier, event-loop lag, RSS/heap/GC, active requests/connections.
- Mongo: operation latency, scanned/returned, plan, pool usage/waits, locks/tickets, cache eviction, replication lag, index hit/size/write cost.
- Queue: enqueue-to-start, duration, lag, retries, dedupe, DLQ, worker CPU/RSS.
- Sockets: connected/rooms/events, delivery/duplicate rate, reconnect, adapter latency.
- External/AI: dependency latency/errors/timeouts/retries; model/prompt version, input/output/cached tokens and cost per tenant/workflow.
- Infrastructure: CPU throttling, memory, network/egress, storage operations, autoscale time, CDN hit rate.

Reporting format

For each run record Git commit, build/image, environment topology, region, data seed/version, test script/version, UTC start/end, warm/cold state, load model, percentiles, throughput, error taxonomy, resource graphs, trace examples, DB plans, pass/fail against SLO, saturation point, cost, and reproducible next action. Never compare runs with different datasets/topologies without flagging the difference.

10. Implementation-ready tasks

Task 1 — Route-split and budget the frontends

- **Business reason:** Current entry bundles delay every user and increase GCC mobile transfer cost.
- **Technical scope:** All seven `src/App.tsx`; Vite configs; heavy import sites for PDF/XLSX/editors/charts/chat.
- **Acceptance criteria:** Each route is a lazy chunk; login/workspace initial JS ≤ 250 kB gzip or an approved staged exception; no static/dynamic duplicate import warnings; skeleton and error boundary present.
- **Testing:** Clean builds, route navigation, offline/slow-4G, visual and auth redirect regression.
- **Dependencies:** CI bundle report.
- **Complexity:** Medium.

Task 2 — Eliminate Admin CSS safelist explosion

- **Business reason:** 10.24 MB CSS is an immediate delivery/build bottleneck.
- **Technical scope:** `admin_dashboard_hr/tailwind.config.ts`, dynamic class producers, semantic color token map.
- **Acceptance criteria:** CSS ≤ 250 kB raw and ≤ 50 kB gzip; all light/dark/status variants render; build ≤ 45 s on reference runner.
- **Testing:** Screenshot matrix of affected components and themes.
- **Dependencies:** Design-approved finite color palette.
- **Complexity:** Small.

Task 3 — Paginate Employee/CRM/task APIs

- **Business reason:** Entire tenant datasets currently cross DB/API/browser boundaries.
- **Technical scope:** `routes/employees.js`, `clientInfoController.js`, canonical task controllers, corresponding list UIs.
- **Acceptance criteria:** Cursor/default 50/max 200; allowlisted sort/filter; minimal fields; stable next cursor; backward-compatible transition; no full fetch for UI pagination.
- **Testing:** Contract tests and 1k/10k/100k synthetic datasets; payload/query targets.
- **Dependencies:** API contract/version decision.

- **Complexity:** Large.

Task 4 — Instrument VirSME end to end

- **Business reason:** Current bottlenecks cannot be safely prioritised or validated in production.
- **Technical scope:** Express/Mongoose/Socket.IO/jobs/frontends/AI; structured logs, metrics, traces and RUM.
- **Acceptance criteria:** One trace covers login/bootstrap and one queued workflow; tenant/request correlation; p50/p95/p99 dashboards; alerts; AI token/cost dashboard; PII redaction tests.
- **Testing:** Synthetic trace, alert and redaction tests.
- **Dependencies:** Telemetry backend and retention policy.
- **Complexity:** Medium.

Task 5 — Introduce revocation-safe auth context cache

- **Business reason:** Auth DB reads multiply every dashboard request.
- **Technical scope:** `auth.js`, `empAuth.js`, `unifiedAuth.js`, grant middleware, shared cache and mutation invalidation.
- **Acceptance criteria:** Cache keys/TTL as defined; logout/permission/session changes invalidate; deny safely when context cannot be verified; no tokens/PII logged.
- **Testing:** DB query counts, p95 benchmark, permission revocation and cross-tenant tests.
- **Dependencies:** Managed Redis/cache, central tenant context.
- **Complexity:** Medium.

Task 6 — Create durable workflow queue and workers

- **Business reason:** In-process jobs duplicate on scale and degrade APIs.
- **Technical scope:** crons, scheduled communication, leave sync/reset, notifications, documents, reports, payroll, AI/email; job schemas/DLQ/idempotency.
- **Acceptance criteria:** Web replicas run no jobs; deterministic idempotency keys; bounded retries/jitter; priority/tenant fairness; DLQ replay; audit trail.
- **Testing:** kill/restart, duplicate enqueue, partial failure, peak queue load.
- **Dependencies:** Queue service and worker deployment.
- **Complexity:** Large.

Task 7 — Object storage and signed file delivery

- **Business reason:** 667 MB local state prevents safe replicas and reliable regional delivery.
- **Technical scope:** upload/download/document routes, `FileStore`, presigned upload, CDN, malware/type controls, migration.
- **Acceptance criteria:** New files never depend on web disk; tenant-prefixed private keys; checksum migration; signed expiry; lifecycle/backup policy.

- **Testing:** access isolation, upload abuse, interrupted multipart, regional latency, restore.
- **Dependencies:** Residency/legal and cloud provider decision.
- **Complexity:** Large.

Task 8 — Consolidate/authenticate Socket.IO

- **Business reason:** Three connection blocks and client-selected rooms are inefficient and unsafe at scale.
- **Technical scope:** `backend/src/index.js`, frontend socket factories, shared adapter.
- **Acceptance criteria:** One handler; JWT handshake; server-derived rooms; per-event schema/size/rate; cross-replica delivery; listener count bounded.
- **Testing:** unauthorised joins, reconnect storm, 1k/5k/10k soak.
- **Dependencies:** Shared adapter/cache.
- **Complexity:** Medium.

Task 9 — Correct AI budgets and routing

- **Business reason:** A 10-token classification request can be configured for at least 2,500 completion tokens.
- **Technical scope:** `deepseekService.js`, watcher classification, model/prompt registry, deterministic gate, cache/idempotency, telemetry.
- **Acceptance criteria:** Per-task token cap; structured output; timeout/fallback; common deterministic intents avoid AI; 100% usage/cost records; sensitive cache policy approved.
- **Testing:** Labeled synthetic evaluation with accuracy/latency/cost thresholds and failure injection.
- **Dependencies:** Approved model selection/data policy.
- **Complexity:** Medium.

Task 10 — Build production-shaped performance CI/staging

- **Business reason:** No automated tests or performance gates exist.
- **Technical scope:** synthetic Mongo seed, contract/integration tests, bundle budgets, smoke benchmark, load scripts, CI and reports.
- **Acceptance criteria:** Clean reproducible pipeline; backend test command passes; performance artifacts retained; threshold regression fails; no production endpoints/data.
- **Testing:** Intentionally introduce and detect bundle/API regression.
- **Dependencies:** CI/staging platform.
- **Complexity:** Medium.

Critical VirSME journey traces

These are static code traces. Timings and query counts are **Measurement required** until instrumentation and seeded staging exist.

Journey	Current path	Verified performance concerns	Validation to add
User login/workspace	React login → /api/auth/login or /api/emp-auth/login → User/Employee, trusted device, attendance/session, JWT → eager app bundle → many dashboard APIs/socket	Large initial JS/CSS; login route performs multiple conditional DB operations; workspace request fan-out; auth DB reads on each follow-up	Browser trace, login spans/query count, cold/warm LCP, 2FA/device branches
Employee onboarding	Admin onboarding UI → /api/onboarding/request-cnic-cv /employee routes → email → complete-profile → uploads → OpenAI CNIC → employee/doc generation	Email/AI/file/PDF work not queued; sensitive local disk; reasoning token floor	Synthetic onboarding end-to-end, retries/idempotency, AI tokens, file scan time
Employee profile retrieval	Profile/employee UI → /api/employees/:id → auth + scoped Employee query/populate shifts	Auth reads precede point query; full employee projection may expose/transfer unnecessary fields	Query/response bytes, projection contract, cache and auth latency
Communication/notification	Connect/CRM → email/chat/WhatsApp APIs and Socket.IO → AssignmentMessage/Chat/WhatsApp models → SMTP/rooms	Huge communication bundle; three socket handlers; in-process schedules; some paths well indexed/paginated	5k socket soak, delivery/duplicate metrics, inbox explain plans, SMTP fault test
Task create/assign/update	Standalone Task → /api/chat/spaces/.../tasks → ChatTask; legacy /api/tasks → mismatched Task model	Duplicate task domains; main task bundle 508 kB; auth/session queries	Canonical contract tests, migration decision, task list query/payload benchmark
CRM lead/customer	CRM UI → /api/client-info → auth/access hierarchy → ClientInfo find/populates	Unbounded lists, live breadth-first hierarchy DB traversal, large CRM bundle	50k-client dataset, cursor/list payload, hierarchy cache correctness
Payroll	Admin/People payroll → payroll period, employees, attendance, loans, salary structures/slips → decryption/PDF/email	Many data sources, serial decryption loops, SalarySlip no index, Puppeteer in request	Employee bands 100/1k/5k, query spans, batch worker time, reconciliation correctness
Expense submission/approval	Header/forms → reimbursement routes → auth → request model/file → unified/open requests → notification	Local file storage, shared request dashboard, unmeasured unified aggregation	Upload/scan duration, approval latency, queue delivery, idempotent status update
Leave request/approval	UI → /api/apply-leave → policy analysis, balances/transactions/history → approval/messages/attendance sync	Complex DB workflow, scheduler sync, several indexes but tenant not first in ApplyLeave schema	Transaction/locking tests, concurrent approvals, policy correctness, p95
Performance reviews	No clearly named end-to-end performance-review model/API/UI was identified in the repository	Capability may be absent, differently named, or incomplete; cannot invent a trace	Product owner maps feature and endpoints; then instrument and benchmark
Dashboard/report	Dashboard effects → many APIs/external weather → employee/attendance/events/sessions; reports → Mongo/Puppeteer/PDF	Duplicate employee calls; no query cache; synchronous report browser	Request count/bytes, bootstrap p95, aggregate plans, async report completion
Search	Header → search endpoints; employees use multi-field regex; chat/message searches use indexed/partially indexed paths	Regex scan/abuse risk; inconsistent scope/pagination	100k/10m datasets, relevance, explain, query length/rate controls
File upload/download	Browser multipart → Multer local disk → Express static or controller download	Local state, 10/20 MB limits, no CDN/object storage/malware evidence	Multipart concurrency, scan, checksum, signed access, GCC TTFB

Journey	Current path	Verified performance concerns	Validation to add
Cross-module workflow	Unified/Open Requests → many request collections → approvals → notifications/payroll/attendance effects	Large aggregation controller, no durable orchestrator, duplicate effect risk	Correlation trace, saga/idempotency tests, failure recovery and audit checks
AI-powered action	Mailbox → image/text prep → OpenAI → parse/fallback → DB/email/doc action	No timeout/usage/cache; forced token floor; deterministic cases still use AI	Evaluation dataset, token/cost/latency, fallback, privacy and approval audit

Significant architectural trade-offs

- **Queue introduction:** adds operational infrastructure and eventual consistency, but is justified for documents, notifications, AI, scheduled messages and payroll batches because retries/idempotency and resource isolation matter more than synchronous completion. User-visible state must show queued/running/failed and preserve audit trails.
- **Auth caching:** reduces DB cost but creates revocation-staleness risk. Keep TTL short, use versioned keys/event invalidation and deny safely; do not remove permission validation.
- **Denormalised dashboard read models:** improve read latency but require reliable event/update logic. Adopt only for aggregates proven expensive; source-of-truth transactions remain normalised.
- **Service decomposition:** separate workloads by scaling/failure characteristics, not organisational fashion. A modular core API plus worker and real-time processes is lower-risk than many business microservices now.
- **Saudi primary region and DR:** same-region app/DB/cache/storage reduces latency and supports residency goals; cross-region replicas improve DR but may conflict with legal/customer commitments and add transfer/consistency cost. Obtain formal residency approval and measure before choosing.
- **Database indexes:** every additional index consumes storage/cache and adds write amplification. Candidate indexes in this report require staging `explain`, observed frequency and write-cost measurement before production creation.

Ten actions with the greatest overall impact

1. Route-split all frontends and enforce initial bundle budgets.
2. Remove the Admin Tailwind safelist explosion.
3. Add server-side cursor pagination, projections and virtualised UI lists for Employee/CRM/task/attendance/request data.
4. Instrument RUM, API/DB/queue/AI traces and per-tenant metrics before further tuning.
5. Move cron, mail, PDF/report, payroll batch and AI work to durable idempotent workers.
6. Move all files to private Saudi/GCC-appropriate object storage with CDN/signed delivery.
7. Cache revocation-safe authentication and stable tenant reference data.
8. Consolidate/authenticate Socket.IO and add a shared adapter before horizontal scaling.
9. Correct AI model/task routing, token caps, deterministic gates, timeouts and cost attribution.

10. Establish production-shaped synthetic staging, automated tests, performance gates, readiness/draining and measured Saudi/GCC deployment architecture.

Audit conclusion: VirSME's breadth is already substantial, but its deployment unit and client delivery are carrying nearly every feature at once. The first performance programme should not begin with speculative microservice or database sharding work. It should begin with verified frontend payload reduction, bounded APIs, observability, durable background execution, stateless storage, and tenant-aware controls. Those changes directly improve user speed, enable reliable GCC scaling, preserve security, and create the evidence needed for later architectural decisions.